

SIMATS ENGINEERING
ASSESSMENT TOOL 1
Case study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1105

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

Code of Conduct:

I DARSHINIE KARANI.V.M (192520033)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Case study

QUESTIONS: 2 Questions

Total Marks:40

1. Smart Hospital Management System

A multi-specialty hospital plans to implement a **Smart Hospital Management System** that enables patients to book appointments, doctors to maintain electronic medical records, laboratories to upload test reports, and pharmacists to manage medicine inventory. Hospital administrators monitor billing, staff, and overall operations.

Question:

Analyze the system and explain how object-oriented principles (**abstraction, encapsulation, inheritance, and polymorphism**) can be applied to model the system. Identify the major classes and relationships, and recommend a suitable **SDLC model** with justification.

2. Smart Banking System

A national bank plans to develop a **Smart Banking System** that supports account creation, fund transfers, online bill payments, loan processing, and transaction monitoring. Customers, bank employees, loan officers, and administrators interact with the system through web and mobile applications.

Question:

Design the system using object-oriented concepts by identifying key classes, objects, and relationships. Explain how object-oriented design improves software quality and recommend the most appropriate **SDLC model** for developing this banking application with suitable justification.

3. Smart Agriculture Monitoring System

A farming organization intends to develop a **Smart Agriculture Monitoring System** that collects soil moisture, temperature, humidity, and weather information using IoT sensors. Farmers receive irrigation recommendations through a mobile application, while agricultural experts analyze crop health using cloud-based analytics.

Question:

Analyze the system requirements and explain how object-oriented principles support modular and reusable software development. Evaluate suitable **SDLC models** and recommend the best model for developing this IoT-based application with justification.

4. Smart Pharmacy Management System

A pharmacy chain plans to implement a **Smart Pharmacy Management System** where customers can search medicines, upload prescriptions, place online orders, and track

deliveries. Pharmacists verify prescriptions, suppliers update stock availability, and administrators manage inventory and billing.

Question:

Analyze the application using object-oriented analysis and design principles. Identify the major system classes and their relationships, explain the benefits of encapsulation and inheritance, and recommend a suitable **SDLC model** with reasons.

5. Smart Transportation Management System

A metropolitan transport authority plans to develop a **Smart Transportation Management System** that enables passengers to book tickets, track buses in real time, receive travel notifications, and make digital payments. Transport operators update schedules, while administrators monitor fleet operations and revenue.

Question:

Apply object-oriented system development concepts to model the system by identifying major classes, relationships, and reusable components. Compare suitable **SDLC models** and recommend the most appropriate model with justification.

6. Smart E-Learning Management System

An educational institution plans to develop a **Smart E-Learning Management System** where students access learning materials, submit assignments, attend online examinations, and receive grades. Faculty members upload course content and evaluate assignments, while administrators manage courses and user accounts.

Question:

Analyze the system using object-oriented design principles and identify the important classes, objects, and relationships required for implementation. Recommend an appropriate **SDLC model** for developing this learning platform and justify your recommendation based on changing academic requirements and continuous feature enhancements.

RUBRICS FOR CALCULATION:

Case study

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

ANSWERS

1. Smart management system

Introduction

The Smart Hospital Management System is designed to manage and automate important hospital activities. The system allows patients to book appointments, doctors to maintain electronic medical records, laboratories to upload test reports, and pharmacists to manage medicine inventory. Hospital administrators can monitor billing, staff, and overall hospital operations.

The system can be developed using Object-Oriented Analysis and Design (OOAD) principles. Object-oriented concepts such as abstraction, encapsulation, inheritance, and polymorphism help create a modular, reusable, secure, and maintainable system.

Object-Oriented Principles

- Abstraction

Abstraction means showing only the important features of an object while hiding unnecessary implementation details.

In the hospital system:

- Patient → patient ID, name, contact details
- Doctor → doctor ID, specialization
- Appointment → date, time, appointment status
- Medical Record → diagnosis, treatment, medical history
- Medicine → medicine name, quantity, price

For example, when a patient books an appointment, the patient only interacts with the booking interface. The internal process of storing and processing the appointment is hidden.

- Encapsulation

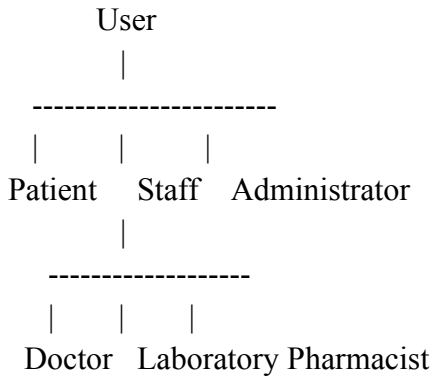
Encapsulation means combining data and methods within a class and restricting direct access to the data.

Only authorized users such as pharmacists can modify medicine stock.

Inheritance

Inheritance allows one class to acquire common properties and methods from another class.

A possible hierarchy is:



For example:

- Staff can contain common attributes such as staffId, name, and contact.
- Doctor, Laboratory, and Pharmacist can inherit these common properties.
- Each subclass can have its own specialized methods.

Polymorphism

Polymorphism means the same method or operation can behave differently depending on the object.

For example:

generateReport()

can have different implementations:

- Doctor → generates medical report
- Laboratory → generates test report
- Administrator → generates hospital operation report

Another example:

update()

- Doctor → updates medical records
- Pharmacist → updates medicine inventory
- Administrator → updates staff information

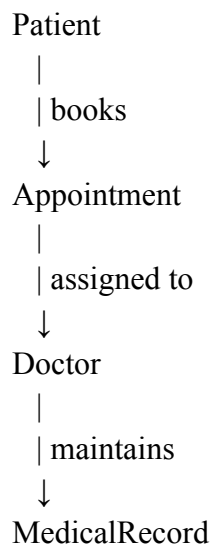
Major Classes

Class	Attributes	Methods
Patient	patientId, name, age, contact	bookAppointment(), viewRecord()
Doctor	doctorId, name, specialization	updateRecord(), prescribeMedicine()

Appointment	appointmentId, date, time, status	book(), cancel(), reschedule()
MedicalRecord	recordId, diagnosis, treatment	addRecord(), updateRecord()
Laboratory	labId, testName, result	uploadReport()
TestReport	reportId, testName, result, date	generateReport()
Pharmacist	pharmacistId, name	manageMedicine(), updateStock()
Medicine	medicineId, name, quantity, price	updateStock(), checkAvailability()
Administrator	adminId, name	manageStaff(), monitorOperations()
Billing	billId, amount, paymentStatus	generateBill(), processPayment()

Relationships Between Classes

The major relationships can be represented as:



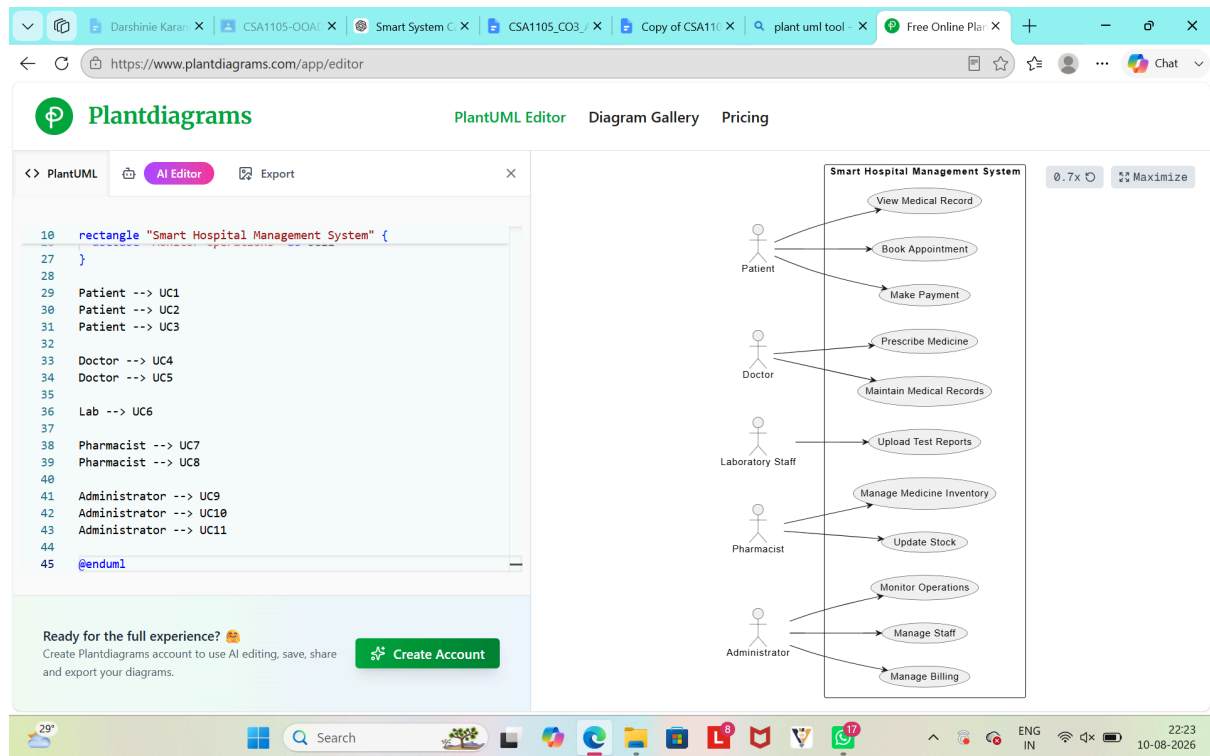
Patient ————— has —————> MedicalRecord

Laboratory ——— uploads ———> TestReport

Pharmacist ——— manages ———> Medicine

Patient ————— pays —————> Billing

Administrator — manages ———> Staff



Relationship Explanation

1. Patient → Appointment
A patient can book one or more appointments.
2. Doctor → Medical Record
Doctors maintain and update patients' medical records.
3. Laboratory → Test Report
Laboratories upload test reports to the system.
4. Pharmacist → Medicine
Pharmacists manage medicine inventory and stock.
5. Patient → Billing
Patients are associated with hospital billing and payments.
6. Administrator → Staff
Administrators manage hospital staff and monitor operations.

Suitable SDLC Model

Recommended Model: Agile SDLC

The Agile model is suitable for developing the Smart Hospital Management System.

Justification

1. Modular development

The system can be divided into different modules:

- Patient Management
- Appointment Management
- Medical Records
- Laboratory Management
- Pharmacy Management
- Billing
- Staff Management

2. Continuous feedback

Doctors, nurses, pharmacists, laboratory staff, and administrators can provide feedback during development.

3. Easy to accommodate changes

Hospital requirements may change during development. Agile allows new requirements to be incorporated into later iterations.

4. Early testing

Each module can be tested after development, helping identify and fix errors early.

5. Incremental development

The system can be developed step-by-step instead of developing the entire system at once.

Example Agile Development

Iteration 1 → Patient & User Management



Iteration 2 → Appointment Management



Iteration 3 → Medical Records



Iteration 4 → Laboratory Management



Iteration 5 → Pharmacy Management



Iteration 6 → Billing & Administration

Advantages of the Proposed System

- Reduces manual hospital work.
- Provides centralized patient information.
- Makes appointment management easier.
- Enables electronic medical records.
- Allows laboratories to upload reports.
- Helps pharmacists manage medicine inventory.

- Improves billing and administrative monitoring.
- Provides better data organization and accessibility.
- Supports modular and reusable software development.

Conclusion

The Smart Hospital Management System can be effectively designed using Object-Oriented Analysis and Design. Abstraction hides unnecessary details, encapsulation protects data, inheritance promotes code reuse, and polymorphism provides flexibility. Major classes include Patient, Doctor, Appointment, MedicalRecord, Laboratory, Pharmacist, Medicine, Administrator, and Billing. Agile SDLC is recommended because the system contains multiple modules and can benefit from continuous feedback, incremental development, and changing requirements.

2. Smart Banking System

Introduction

The Smart Banking System is designed to provide secure and convenient banking services through web and mobile applications. The system supports account creation, fund transfers, online bill payments, loan processing, and transaction monitoring. Customers, bank employees, loan officers, and administrators interact with the system to perform different banking activities.

Object-Oriented Analysis and Design (OOAD) can be used to develop this system by representing banking entities as objects and organizing them into classes with defined attributes, methods, and relationships.

Major Classes

The important classes in the Smart Banking System are:

Class	Attributes	Methods
Customer	customerId, name, email, phone	createAccount(), transferFund(), payBill()
Account	accountNo, accountType, balance	deposit(), withdraw(), checkBalance()
Transaction	transactionId, amount, date, type	processTransaction(), viewTransaction()
BillPayment	billId, billType, amount, dueDate	makePayment(), checkStatus()
Loan	loanId, loanAmount, loanType, status	applyLoan(), updateStatus()

LoanOfficer	employeeId, name, department	processLoan(), approveLoan()
BankEmployee	employeeId, name, role	manageCustomer(), updateAccount()
Administrator	adminId, name	manageUsers(), monitorSystem()

Objects

An object is an instance of a class.

For example:

Customer Object

Customer

customerId = C101

name = "Arun"

email = "arun@email.com"

Account Object

Account

accountNo = A1001

accountType = "Savings"

balance = ₹50,000

Loan Object

Loan

loanId = L501

loanAmount = ₹5,00,000

loanType = "Education"

status = "Pending"

These objects interact with one another to perform banking operations.

Relationships Between Classes

The major relationships are:

Customer — owns —> Account

Customer — performs —> Transaction

Customer — makes —> BillPayment

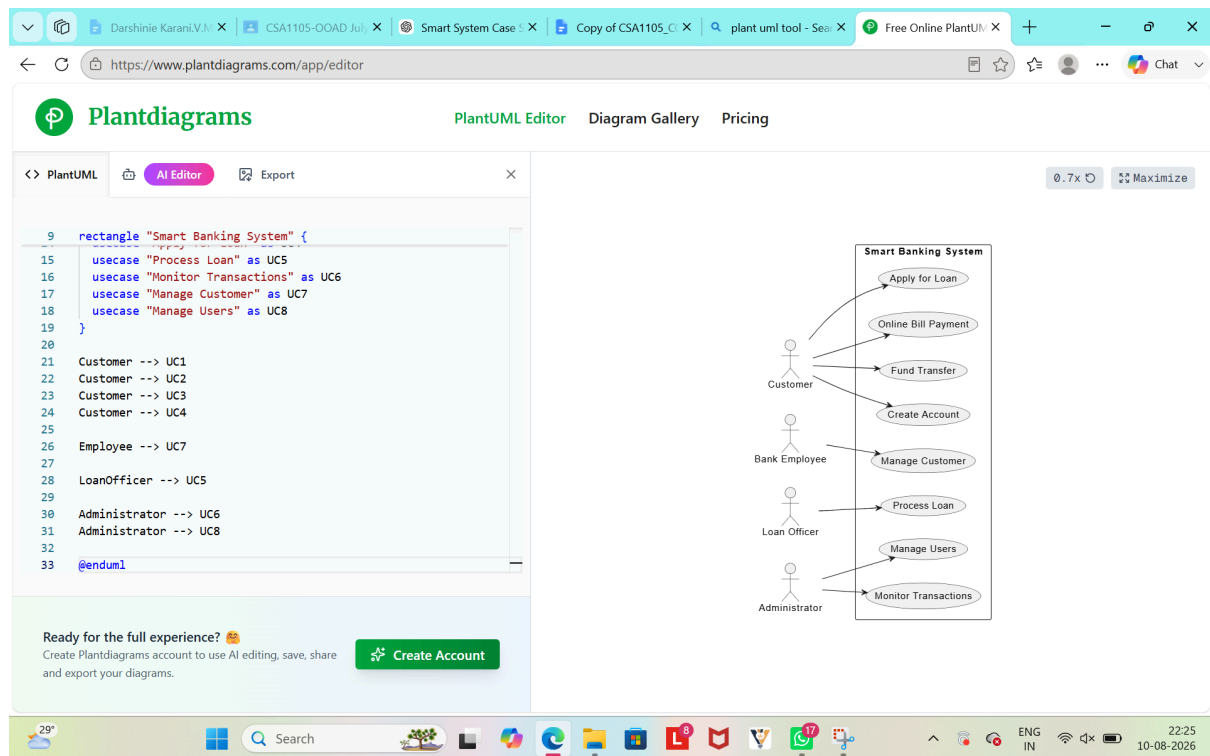
Customer — applies —> Loan

LoanOfficer — processes —> Loan

BankEmployee — manages —> Customer

Administrator — manages —> Users

Account — has —> Transaction



Relationship Explanation

1. Customer – Account
A customer can own one or more bank accounts.
2. Customer – Transaction
Customers perform transactions such as fund transfers and payments.
3. Customer – Bill Payment
Customers can make online payments for different bills.
4. Customer – Loan
Customers can apply for loans through the system.

5. Loan Officer – Loan
Loan officers process and evaluate loan applications.
6. Bank Employee – Customer
Bank employees manage customer information and account-related activities.
7. Administrator – Users
Administrators manage users and monitor the banking system.
8. Account – Transaction
Each account can have multiple transactions.

Object-Oriented Concepts

Abstraction

Abstraction hides unnecessary implementation details and shows only the required banking operations.

For example, when a customer performs a fund transfer, the customer only needs to enter the recipient and amount. The internal process of authentication, validation, and transaction processing is hidden.

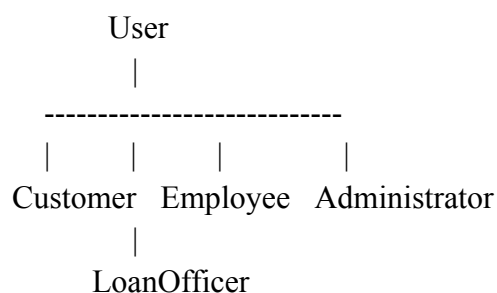
Encapsulation

Encapsulation protects sensitive banking information by keeping data and related methods inside a class.

The account balance should not be directly modified by users. It should be changed through authorized methods such as deposit() and withdraw().

Inheritance

Inheritance allows classes to reuse common properties and methods.



For example, common attributes such as userId, name, email, and login details can be inherited from the User class.

Polymorphism

Polymorphism allows the same method to perform different operations depending on the object.

For example:

processPayment()

can be implemented differently for:

- Fund transfer
- Bill payment
- Loan repayment

Similarly, a *generateReport()* method can generate different reports for a bank employee and an administrator.

How Object-Oriented Design Improves Software Quality

1. Reusability

Classes and methods can be reused in different parts of the banking application.

2. Security

Encapsulation protects sensitive information such as account balances and transaction details.

3. Maintainability

Each banking function is separated into different classes, making the system easier to maintain.

4. Modularity

The system can be divided into modules such as:

- Account Management
- Fund Transfer
- Bill Payment
- Loan Management
- Transaction Management
- User Management

5. Scalability

New banking features can be added without completely redesigning the existing system.

6. Flexibility

Polymorphism allows different banking operations to use common interfaces while having different implementations.

7. Reduced Complexity

Abstraction hides complex internal processes from users and developers.

Recommended SDLC Model

Agile Model

The Agile SDLC model is the most appropriate model for developing the Smart Banking System.

Justification

1. Changing Requirements

Banking applications frequently require new features and improvements. Agile allows requirements to be changed during development.

2. Incremental Development

The system can be developed in small iterations.

Iteration 1 → Account Creation



Iteration 2 → Fund Transfer



Iteration 3 → Online Bill Payment



Iteration 4 → Loan Processing



Iteration 5 → Transaction Monitoring



Iteration 6 → Security & Final Integration

3. Continuous Testing

Each feature can be tested immediately after development. This is particularly useful for detecting errors in financial transactions.

4. Customer Feedback

Bank employees and customers can provide feedback during development, allowing the system to be improved continuously.

5. Easy Feature Enhancement

New services and features can be incorporated into future iterations without affecting the entire system.

6. Risk Reduction

Security, transaction, and performance issues can be identified and addressed during individual iterations instead of waiting until the end of development.

Advantages of Smart Banking System

- Provides convenient online banking services.
- Supports account creation.
- Enables quick fund transfers.
- Allows online bill payments.
- Simplifies loan processing.
- Helps monitor transactions.
- Reduces manual banking work.
- Improves data organization.
- Provides better accessibility through web and mobile applications.
- Supports secure and maintainable software development.

Conclusion

The Smart Banking System can be effectively designed using Object-Oriented Analysis and Design. Classes such as Customer, Account, Transaction, Loan, BillPayment, LoanOfficer, BankEmployee, and Administrator represent the major components of the system.

The use of abstraction, encapsulation, inheritance, and polymorphism improves security, reusability, maintainability, modularity, and scalability. Among the SDLC models, Agile is the most suitable because it supports incremental development, continuous testing, customer feedback, and changing banking requirements.